CENTRO PAULA SOUZA
FACULDADE DE TECNOLOGIA DE FRANCA
"Dr. THOMAZ NOVELINO"
TECNOLOGIA EM DESENVOLVIMENTO SOFTWARE MULTIPLATAFORMA

DOUGLAS ANDRADE CINTRA DE BRITO
ELEONORA CINTRA CASTRO
JOÃO PEDRO RACERO SANTOS
RYAN FILIPE DAMASCENO
THIAGO VERONEZ DA SILVA SANTOS

POUPEE – ORGANIZADOR DE FINANÇAS

Requisitos, Modelagem, Implementação e Implantação

FRANCA/SP
2026

1 Introdução

Boa parte de nossa vida é direcionada pelo ganho ou pela perda de dinheiro, seja no momento em que buscamos trabalho, seja na hora de pedir um empréstimo no banco para alcançar um objetivo. Tudo isso precisa ser organizado, pois a má gestão dos recursos financeiros acarreta dívidas e, possivelmente, restrições cadastrais. Grande parte das pessoas não sabe, ao final do mês, para onde foi parte de sua renda: os gastos se acumulam, contas vencem sem serem percebidas e o controle em planilhas acaba abandonado por falta de praticidade. A dor do usuário, portanto, é não dispor de uma forma simples e rápida de registrar o que ganha e o que gasta e de enxergar, em um só lugar, quanto já entrou, quanto ainda precisa pagar e para onde o dinheiro está indo, o que leva ao esquecimento de contas, ao pagamento de juros por atraso e ao endividamento.

O objetivo deste trabalho é o desenvolvimento de uma aplicação WEB, denominada Poupee, com a finalidade de ajudar pessoas físicas a organizar suas finanças pessoais, permitindo que registrem receitas e despesas vinculadas a categorias, acompanhem quais contas estão pendentes e quais já foram pagas e visualizem, em um único painel, o balanço do período com o saldo líquido e a distribuição dos gastos por categoria. O trabalho compreende a análise de requisitos, a modelagem do sistema (BPMN, casos de uso e modelo entidade-relacionamento), a implementação da aplicação, com front-end, API REST e banco de dados relacional PostgreSQL, e a sua publicação na internet, onde pode ser usada por qualquer pessoa a partir do navegador.

A metodologia adotada compreendeu: (i) análise de mercado e pesquisa bibliográfica sobre gestão de finanças pessoais; (ii) identificação dos atores e escrita das histórias de usuário; (iii) modelagem do processo de negócio em BPMN; (iv) levantamento e classificação dos requisitos funcionais, não funcionais e das regras de negócio; (v) descrição dos casos de uso, com cenários principal e alternativos, e elaboração do diagrama de casos de uso; (vi) modelagem do diagrama entidade-relacionamento; (vii) construção do protótipo navegável em HTML, CSS e JavaScript; (viii) implementação da API REST em Node.js e do banco de dados relacional PostgreSQL; (ix) integração das telas à API; e (x) publicação do sistema na Vercel, com versionamento no GitHub e testes automatizados.

A contribuição do trabalho está em traduzir um problema cotidiano — a dificuldade de controlar as próprias finanças — em requisitos claros e rastreáveis, regras de negócio explícitas e um sistema web funcional, publicado na internet, que implementa essas regras. Para o usuário final, o Poupee contribui como uma alternativa simples às planilhas e aos aplicativos com excesso de telas e formulários, reunindo em um só lugar o registro dos lançamentos e a visão do balanço. Do ponto de vista acadêmico, o projeto exercita a aplicação prática das técnicas de levantamento e especificação de requisitos, modelagem de processos em BPMN, descrição de casos de uso, modelagem de dados, prototipação de interfaces, desenvolvimento back-end com banco de dados relacional e implantação em nuvem.

Com este projeto, espera-se entregar uma especificação completa e coerente do Poupee e um sistema funcional que a implementa, permitindo registrar um lançamento em poucos segundos, evitar o esquecimento de contas a vencer, mostrar quanto é gasto em cada categoria e dar uma visão clara do saldo do período, favorecendo decisões financeiras mais conscientes e reduzindo o risco de endividamento.

2 Levantamento de Requisitos

O levantamento de requisitos é essencial no desenvolvimento de software, pois define com clareza o que o sistema deve fazer antes do início da programação. Essa etapa evita o retrabalho, reduz custos e garante que o produto final atenda às necessidades reais dos usuários de forma simples e direta. Requisitos mal compreendidos geram sistemas que não resolvem o problema para o qual foram criados; por isso, investir tempo nessa fase alinha as expectativas entre a equipe e o público-alvo e permite planejar o projeto com maior segurança.

2.1 Elicitação e especificação dos Requisitos

A elicitação dos requisitos do Poupee foi realizada por meio da combinação das seguintes técnicas:

- **Análise de mercado e benchmarking –** estudo de soluções existentes de controle financeiro pessoal (aplicativos e planilhas), identificando as funcionalidades essenciais, os pontos fortes e as principais reclamações dos usuários, como excesso de telas e formulários complicados.

- **Pesquisa bibliográfica –** leitura de matérias e artigos sobre gestão de finanças pessoais e educação financeira, que orientaram as regras de negócio (data de competência, balanço mensal, classificação por categoria).
- **Brainstorming com a equipe –** reuniões do grupo para priorizar as funcionalidades e definir o escopo mínimo viável do produto (MVP).
- **Entrevistas informais e questionário –** conversas com colegas e familiares sobre como controlam (ou não) seus gastos, para levantar dores reais, como esquecer contas a vencer e não saber em que categoria o dinheiro é mais gasto.
- **Prototipação –** construção de um protótipo navegável usado para validar visualmente o fluxo de uso e refinar os requisitos a partir do retorno dos usuários.

O ator identificado foi:

- **Usuário –** pessoa física que deseja organizar suas finanças. Cria sua conta, cadastra categorias, registra despesas e receitas, marca contas como pagas e consulta o balanço do período. Por se tratar de um sistema de uso individual, não há distinção de perfis administrativos.

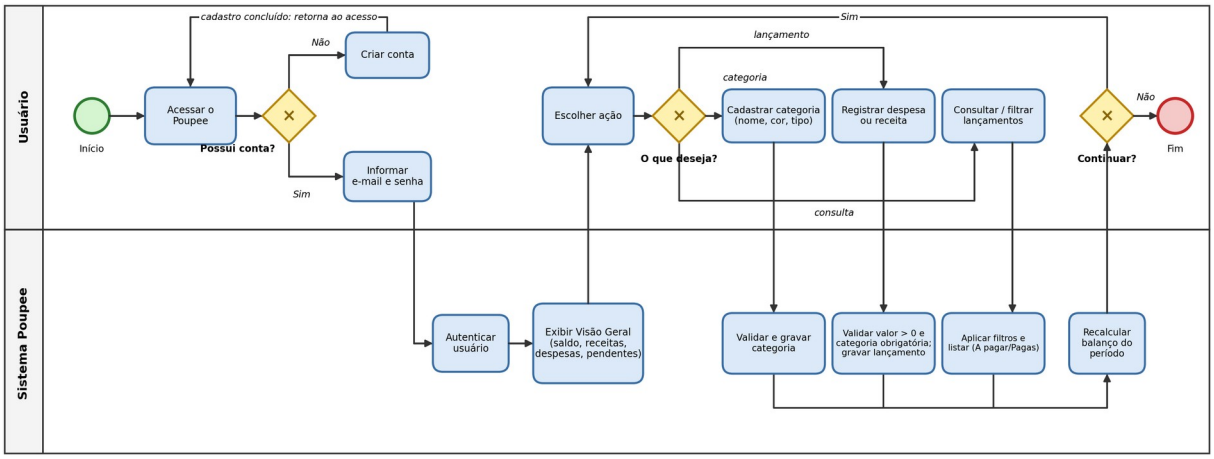
As especificações foram registradas na forma de histórias de usuário:

- Como usuário, quero criar uma conta com nome, e-mail e senha.
- Como usuário, quero entrar no sistema com e-mail e senha, para acessar minhas informações financeiras com segurança.
- Como usuário, quero cadastrar uma despesa informando descrição, valor, data de vencimento, categoria e status (a pagar ou paga) e poder consultá-la, alterá-la ou excluí-la, para não esquecer nenhuma conta.
- Como usuário, quero cadastrar uma receita informando descrição, valor, data de recebimento e categoria e poder consultá-la, alterá-la ou excluí-la, para saber quanto dinheiro entrou no período.
- Como usuário, quero listar minhas despesas filtrando por "A pagar" e "Pagas" e por mês/ano, para controlar rapidamente o que ainda preciso quitar.
- Como usuário, quero visualizar o saldo do período (receitas menos despesas pagas) e os gastos por categoria em um dashboard, para entender para onde meu dinheiro está indo.
- Como usuário, quero criar, consultar, alterar e excluir minhas próprias categorias com nome e cor, para classificar os lançamentos do jeito que faz sentido para mim.

2.2 BPMN

O processo de negócio modelado em BPMN (Figura 1) inicia quando o usuário acessa o Poupee. Caso ainda não possua conta, ele realiza o cadastro e, concluído o cadastro, o fluxo retorna à etapa de acesso, na qual o usuário passa a informar e-mail e senha; caso já possua conta, informa e-mail e senha diretamente. O sistema autentica o usuário e exibe a Visão Geral, com o saldo do período, os totais de receitas e despesas e as contas pendentes. A partir daí o usuário escolhe uma ação: cadastrar uma categoria, registrar uma despesa ou receita, ou consultar e filtrar seus lançamentos. Em cada caso o sistema valida os dados — valor positivo e categoria obrigatória, conforme as regras de negócio — grava as informações e recalcula o balanço do período. O ciclo se repete enquanto o usuário desejar continuar utilizando o sistema.

Figura 1 – Diagrama BPMN do processo de controle financeiro



Fonte: elaborado pelos autores (2026).

2.3 Requisitos Funcionais

O Quadro 1 lista os requisitos funcionais identificados, classificados quanto à categoria (oculto ou evidente ao usuário) e à prioridade.

Quadro 1 – Requisitos Funcionais do sistema

RF001 – Manter Usuário	**Categoria:** ( ) Oculto (X) Evidente	**Prioridade:** (X) Altíssima ( ) Alta ( ) Média ( ) Baixa
Descrição: O sistema deve permitir manter (incluir, consultar, deletar e alterar) os dados do usuário: (i) incluir – criar a conta informando nome completo, e-mail e senha (com confirmação), para que seus dados financeiros fiquem vinculados a ela; (ii) consultar – visualizar os dados da própria conta (nome, e-mail e tema); (iii) alterar – modificar o nome e o tema (claro ou escuro) da conta.		

RF002 – Manter Despesa	**Categoria:** ( ) Oculto (X) Evidente	**Prioridade:** (X) Altíssima ( ) Alta ( ) Média ( ) Baixa
Descrição: O sistema deve permitir que o usuário insira uma nova despesa informando descrição, valor, data de vencimento, categoria e status de liquidação (a pagar ou já paga), bem como consultar, alterar e excluir despesas existentes.		
RF003 – Manter Receita	**Categoria:** ( ) Oculto (X) Evidente	**Prioridade:** (X) Altíssima ( ) Alta ( ) Média ( ) Baixa
Descrição: O sistema deve permitir que o usuário insira uma nova receita informando descrição, valor, data de recebimento e categoria, bem como consultar, alterar e excluir receitas existentes, listando-as com filtro por período e busca por descrição e permitindo a exportação em CSV e PDF.		
RF004 – Listar Despesas	**Categoria:** ( ) Oculto (X) Evidente	**Prioridade:** ( ) Altíssima (X) Alta ( ) Média ( ) Baixa
Descrição: O sistema deve exibir a lista de despesas cadastradas, permitindo filtrar por status ("Todas", "A pagar" e "Pagas"), por mês/ano e por texto da descrição, exibindo o total filtrado e permitindo a exportação em CSV e PDF.		
RF005 – Exibir Dashboard Financeiro (Visão Geral)	**Categoria:** ( ) Oculto (X) Evidente	**Prioridade:** (X) Altíssima ( ) Alta ( ) Média ( ) Baixa
Descrição: O sistema deve calcular e exibir, em um dashboard, para o período selecionado, o saldo do período (total de receitas menos despesas pagas), o saldo previsto (total de receitas menos todas as despesas, inclusive as ainda não pagas), a quantidade e o valor das contas pendentes, o gráfico de gastos por categoria e os lançamentos recentes.		
RF006 – Manter Categoria	**Categoria:** ( ) Oculto (X) Evidente	**Prioridade:** ( ) Altíssima (X) Alta ( ) Média ( ) Baixa
Descrição: O sistema deve permitir que o usuário crie, consulte, altere e exclua categorias tanto para despesas quanto para receitas, informando nome, cor e tipo (ex.: Moradia, Alimentação, Lazer, Renda Fixa), além de marcar favoritas e reordená-las.		

RF007 – Alterar Status da Despesa	Categoria: ( ) Oculto (X) Evidente	Prioridade: ( ) Altíssima ( ) Alta (X) Média ( ) Baixa
Descrição: O sistema deve permitir que o usuário marque uma despesa como paga ou a pagar diretamente na listagem, atualizando imediatamente o total de contas pendentes e o balanço.		

Fonte: elaborado pelos autores (2026).

2.4 Requisitos Não Funcionais

O Quadro 2 apresenta os requisitos não funcionais, que definem as qualidades e restrições sob as quais o sistema deve operar.

Quadro 2 – Requisitos Não Funcionais do sistema

RNF001 – Aplicação Web	**Categoria:** Organizacional O sistema deve ser uma aplicação web acessível através de qualquer navegador moderno, sem necessidade de instalação.	**Tipo** ( ) Desejável (X) Obrigatório (X) Permanente ( ) Transitório
RNF002 – Usabilidade da Interface	**Categoria:** Produto O sistema deve possuir uma interface intuitiva, limpa e de fácil navegação, permitindo que usuários de todas as idades utilizem a aplicação sem dificuldades e registrem um lançamento em poucos segundos.	**Tipo** ( ) Desejável (X) Obrigatório (X) Permanente ( ) Transitório
RNF003 – Armazenamento de Dados	**Categoria:** Organizacional O sistema deve utilizar um banco de dados relacional (PostgreSQL, hospedado no Neon) para armazenar de forma persistente apenas as informações essenciais: dados básicos do usuário, categorias e lançamentos.	**Tipo** ( ) Desejável (X) Obrigatório (X) Permanente ( ) Transitório
RNF004 – Cores do Sistema	**Categoria:** Produto (Design de Interface) O sistema deve seguir a identidade visual da marca Poupee, utilizando a cor laranja (#F2600C) como tom predominante nos elementos principais da interface (botões de ação principal, cabeçalhos e destaques de navegação).	**Tipo** ( ) Desejável (X) Obrigatório (X) Permanente ( ) Transitório
RNF005 – Segurança e Autenticação	**Categoria:** Produto (Segurança) O acesso às funcionalidades do sistema deve ocorrer somente após a autenticação do usuário por e-mail e senha; as senhas devem ser armazenadas de forma criptografada (hash) e o acesso aos dados de um usuário deve ser restrito à sua própria conta.	**Tipo** ( ) Desejável (X) Obrigatório (X) Permanente ( ) Transitório

RNF006 – Responsividade	**Categoria:** Produto A interface deve se adaptar a telas de computadores, tablets e smartphones, com menu lateral recolhível em dispositivos móveis.	**Tipo** (X) Desejável ( ) Obrigatório (X) Permanente ( ) Transitório
RNF007 – Tema Claro e Escuro	**Categoria:** Produto O sistema deve oferecer alternância entre tema claro e escuro, mantendo a preferência do usuário.	**Tipo** (X) Desejável ( ) Obrigatório (X) Permanente ( ) Transitório

Fonte: elaborado pelos autores (2026).

2.5 Regras de Negócio

O Quadro 3 apresenta as regras de negócio que condicionam o funcionamento do sistema.

Quadro 3 – Regras de Negócio do sistema

RN001 – Cálculo do Balanço do Período
Descrição: O saldo do período deve ser calculado subtraindo do total das receitas o valor total das despesas já pagas, registradas dentro do mesmo período (mês/ano) selecionado. O saldo previsto considera também as despesas ainda não pagas, isto é, é o total das receitas menos o total de todas as despesas do período.
RN002 – Agrupamento de Lançamentos
Descrição: As receitas e despesas cadastradas devem ser vinculadas obrigatoriamente à sua data de competência (vencimento, para despesas; recebimento, para receitas), permitindo o agrupamento automático dos lançamentos por mês e ano.
RN003 – Restrição de Valores
Descrição: O sistema só deve permitir o cadastro de receitas e despesas com valores numéricos positivos e maiores que zero.
RN004 – Classificação por Categoria
Descrição: Toda receita ou despesa cadastrada deve ser obrigatoriamente vinculada a uma categoria específica, para permitir o agrupamento e a visualização dos gastos segmentados.
RN005 – Compatibilidade de Tipo
Descrição: A categoria escolhida para um lançamento deve ser do mesmo tipo do lançamento: despesas só podem usar categorias de despesa e receitas só podem usar categorias de receita.
RN006 – Status dos Lançamentos
Descrição: Somente despesas possuem status de liquidação ("A pagar" ou "Paga"). Receitas são sempre consideradas recebidas na data informada.
RN007 – Unicidade do E-mail
Descrição: Cada e-mail pode estar vinculado a apenas uma conta de usuário no sistema.

RN008 – Exclusão de Categoria
Descrição: Uma categoria só pode ser excluída se não possuir lançamentos vinculados; caso contrário, o sistema deve impedir a exclusão e informar o usuário.

Fonte: elaborado pelos autores (2026).

2.6 Casos de Uso

A partir dos requisitos funcionais foram identificados sete casos de uso, descritos a seguir conforme o modelo da disciplina. Os casos de uso Manter Despesa e Manter Receita são estendidos por Manter Categoria quando a categoria desejada ainda não existe, e Listar Despesas é estendido por Alterar Status da Despesa; o ponto de extensão e o relacionamento «extend» estão descritos nos cenários de cada caso de uso. Os casos de uso Manter Despesa, Manter Receita e Manter Categoria contemplam as quatro operações de um CRUD (incluir, consultar, alterar e excluir), enquanto Manter Usuário contempla incluir, consultar e alterar, pois a exclusão de conta não faz parte do escopo do sistema. Com exceção da inclusão de usuário, todos os casos de uso têm como pré-condição que o usuário esteja autenticado no sistema, mediante e-mail e senha (RNF005).

Quadro 4 – Use Case Manter Usuário

Caso de Uso – Manter Usuário	
ID	UC001
Descrição	Este caso de uso tem por objetivo manter (incluir, consultar e alterar) a conta do usuário no Poupee, permitindo que uma pessoa passe a utilizar o sistema e gerencie seus dados cadastrais.
Ator Primário	Usuário
Atores Secundários	—
Pré-condição	Para incluir: o usuário não possui conta cadastrada e está na tela inicial (landing page) ou na tela de login. Para consultar ou alterar: o usuário está autenticado no sistema.
Cenário Principal	1. O use case inicia (inclusão) quando o usuário seleciona a opção "Criar conta" ou "Começar grátis". 2. O sistema carrega o formulário de criação de conta. 3. O usuário informa nome completo, e-mail, senha e confirmação de senha. 4. O usuário confirma a criação da conta. 5. O sistema valida os dados informados (campos obrigatórios, formato do e-mail, senhas iguais e e-mail ainda não cadastrado – RN007). 6. O sistema gera o código do usuário, grava a senha de forma criptografada (RNF005) e cria as categorias padrão da conta. 7. O sistema exibe mensagem de sucesso e retorna à tela de acesso (login),

	na qual o usuário informa e-mail e senha para entrar no sistema.
Pós-condição	Um novo usuário foi cadastrado e pode acessar o sistema com e-mail e senha. Nos cenários alternativos, os dados do usuário foram consultados ou alterados.
Cenário Alternativo	1a – Consulta (pesquisa) dos dados do usuário 1a.1 O usuário autenticado seleciona a opção "Minha conta". 1a.2 O sistema exibe o nome e o e-mail cadastrados e o caso de uso termina. 1b – Alteração dos dados do usuário 1b.1 Na tela "Minha conta", o usuário aciona a opção "Editar". 1b.2 O sistema carrega o formulário preenchido com os dados atuais. 1b.3 O usuário altera o nome e/ou o tema (claro ou escuro) e confirma; o sistema valida os dados (nome obrigatório), grava as alterações e informa o sucesso da operação. 1c – Usuário já possui conta 1c.1 O usuário aciona o link "Já tem conta? Entrar". 1c.2 O sistema exibe a tela de login e este caso de uso termina. 5a – E-mail já cadastrado 5a.1 O sistema informa que já existe uma conta com o e-mail informado e retorna ao passo 3. 5b – Senhas não conferem ou campos obrigatórios em branco 5b.1 O sistema destaca os campos com erro e retorna ao passo 3.

Fonte: elaborado pelos autores (2026).

Quadro 5 – Use Case Manter Despesa

Caso de Uso – Manter Despesa	
ID	UC002
Descrição	Este caso de uso tem por objetivo manter (incluir, consultar, alterar e excluir) as despesas do usuário, indicando se já foram pagas ou ainda estão pendentes. Quando a categoria desejada ainda não existe, este caso de uso é estendido por Manter Categoria (UC006), por meio do relacionamento «extend».
Ator Primário	Usuário
Atores Secundários	—
Pré-condição	O usuário está autenticado no sistema e existe ao menos uma categoria de despesa cadastrada.
Cenário Principal	1. O use case inicia quando o usuário seleciona a opção "Novo lançamento" ou "Nova despesa". 2. O sistema carrega o formulário de lançamento com o tipo "Despesa" selecionado e gera automaticamente o código do lançamento. 3. O usuário informa a descrição, o valor e a data de vencimento. 4. O sistema apresenta as categorias de despesa disponíveis (RN005) e o usuário seleciona uma delas. [Ponto de extensão: ver cenário 4a.] 5. O usuário informa o status da despesa: "A pagar" ou "Já paga".

	6. O usuário confirma o cadastro. 7. O sistema valida os dados informados (campos obrigatórios, valor maior que zero – RN003, categoria obrigatória – RN004). 8. O sistema grava a despesa, recalcula o balanço do período (RN001) e exibe mensagem de sucesso. 9. O sistema retorna à tela de origem (Visão Geral ou lista de Despesas) e o caso de uso termina.
Pós-condição	Uma despesa foi incluída, consultada, alterada ou excluída e, quando houve modificação, o balanço do período foi atualizado.
Cenário Alternativo	1a – Consulta (pesquisa) de despesa 1a.1 O usuário digita parte da descrição no campo de busca ou seleciona uma despesa na lista de despesas (UC004). 1a.2 O sistema exibe os dados da despesa (descrição, valor, vencimento, categoria e status) e o caso de uso termina. 1b – Alteração de despesa existente 1b.1 O usuário seleciona uma despesa na lista e aciona a opção "Editar". 1b.2 O sistema carrega o formulário preenchido com os dados atuais. 1b.3 O usuário altera um ou mais campos e confirma; o caso de uso continua no passo 7. 1c – Exclusão de despesa 1c.1 O usuário seleciona uma despesa na lista e aciona "Excluir". 1c.2 O sistema solicita confirmação; se confirmado, exclui a despesa, recalcula o balanço e o caso de uso termina. 4a – Categoria desejada não existe (extend: Manter Categoria) 4a.1 O usuário aciona "Nova categoria"; neste ponto de extensão, o sistema executa o caso de uso Manter Categoria (UC006), relacionamento «extend», no cenário de inclusão. 4a.2 Ao retornar, o sistema seleciona a nova categoria e o caso de uso continua no passo 5. 6a – Usuário cancela o lançamento 6a.1 O sistema descarta os dados e o caso de uso termina. 7a – Valor inválido (zero, negativo ou não numérico) 7a.1 O sistema informa que só são aceitos valores positivos maiores que zero e retorna ao passo 3. 7b – Campos obrigatórios não preenchidos 7b.1 O sistema destaca os campos obrigatórios e retorna ao passo 3.

Fonte: elaborado pelos autores (2026).

Quadro 6 – Use Case Manter Receita

Caso de Uso – Manter Receita	
ID	UC003
Descrição	Este caso de uso tem por objetivo manter (incluir, consultar, alterar e excluir) as receitas do usuário, registrando o que entrou em seu caixa. Quando a categoria desejada ainda não existe, este caso de uso é estendido por Manter Categoria (UC006), por meio do relacionamento «extend».
Ator Primário	Usuário

Atores Secundários	—
Pré-condição	O usuário está autenticado no sistema e existe ao menos uma categoria de receita cadastrada.
Cenário Principal	1. O use case inicia quando o usuário seleciona a opção "Novo lançamento" ou "Nova receita". 2. O sistema carrega o formulário de lançamento com o tipo "Receita" selecionado e gera automaticamente o código do lançamento. 3. O usuário informa a descrição, o valor e a data de recebimento. 4. O sistema apresenta as categorias de receita disponíveis (RN005) e o usuário seleciona uma delas. [Ponto de extensão: ver cenário 4a.] 5. O usuário confirma o cadastro. 6. O sistema valida os dados informados (campos obrigatórios, valor maior que zero – RN003, categoria obrigatória – RN004). 7. O sistema grava a receita com situação "Recebida" (RN006), recalcula o balanço do período (RN001) e exibe mensagem de sucesso. 8. O sistema retorna à tela de origem e o caso de uso termina.
Pós-condição	Uma receita foi incluída, consultada, alterada ou excluída e, quando houve modificação, o balanço do período foi atualizado.
Cenário Alternativo	1a – Consulta (pesquisa) de receita 1a.1 O usuário digita parte da descrição no campo de busca ou seleciona uma receita na lista de receitas. 1a.2 O sistema exibe os dados da receita (descrição, valor, data de recebimento e categoria) e o caso de uso termina. 1b – Alteração de receita existente 1b.1 O usuário seleciona uma receita na lista e aciona "Editar". 1b.2 O sistema carrega o formulário preenchido; o usuário altera os campos e confirma; o caso de uso continua no passo 6. 1c – Exclusão de receita 1c.1 O usuário seleciona uma receita e aciona "Excluir"; o sistema solicita confirmação e, se confirmado, exclui a receita, recalcula o balanço e o caso de uso termina. 4a – Categoria desejada não existe (extend: Manter Categoria) 4a.1 O usuário aciona "Nova categoria"; neste ponto de extensão, o sistema executa o caso de uso Manter Categoria (UC006), relacionamento «extend», no cenário de inclusão, e retorna ao passo 5. 5a – Usuário cancela o lançamento 5a.1 O sistema descarta os dados e o caso de uso termina. 6a – Valor inválido 6a.1 O sistema informa que só são aceitos valores positivos maiores que zero e retorna ao passo 3. 6b – Campos obrigatórios não preenchidos 6b.1 O sistema destaca os campos obrigatórios e retorna ao passo 3.

Fonte: elaborado pelos autores (2026).

Quadro 7 – Use Case Listar Despesas

Caso de Uso – Listar Despesas

ID	UC004
Descrição	Este caso de uso tem por objetivo apresentar ao usuário todas as suas despesas, com filtros por status, período e descrição, permitindo consultar os detalhes de cada lançamento e exportar a lista. É estendido por Alterar Status da Despesa (UC007), por meio do relacionamento «extend».
Ator Primário	Usuário
Atores Secundários	—
Pré-condição	O usuário está autenticado no sistema.
Cenário Principal	1. O use case inicia quando o usuário seleciona a opção "Despesas" no menu lateral. 2. O sistema carrega a lista de despesas do período selecionado, ordenada por data, exibindo descrição, categoria, vencimento, valor e status. 3. O sistema exibe o total filtrado e as opções de filtro "Todas", "A pagar" e "Pagas". 4. O usuário seleciona um filtro de status e/ou um período (mês e ano) e/ou digita parte da descrição. 5. O sistema aplica os filtros (RN002) e atualiza a lista e o total filtrado. 6. O usuário consulta os detalhes de uma despesa ou aciona as ações disponíveis (editar, excluir, marcar como paga). [Ponto de extensão: ver cenário 6a.] 7. O caso de uso termina quando o usuário navega para outra tela.
Pós-condição	As despesas foram apresentadas conforme os filtros escolhidos.
Cenário Alternativo	2a – Não há despesas no período 2a.1 O sistema exibe a mensagem "Nenhuma despesa encontrada" e o atalho para cadastrar uma nova despesa (UC002). 3a – Usuário solicita exportação 3a.1 O usuário aciona "Exportar CSV" ou "Exportar PDF". 3a.2 O sistema gera o arquivo com as despesas filtradas e o disponibiliza para download; o caso de uso continua no passo 4. 6a – Usuário altera o status de uma despesa (extend: Alterar Status da Despesa) 6a.1 Neste ponto de extensão, o sistema executa o caso de uso Alterar Status da Despesa (UC007), relacionamento «extend», e retorna ao passo 5. 6b – Usuário edita ou exclui uma despesa 6b.1 O sistema executa o caso de uso Manter Despesa (UC002) nos cenários de alteração ou exclusão e retorna ao passo 5.

Fonte: elaborado pelos autores (2026).

Quadro 8 – Use Case Exibir Dashboard Financeiro

Caso de Uso – Exibir Dashboard Financeiro	
ID	UC005
Descrição	Este caso de uso tem por objetivo apresentar, em um dashboard (Visão Geral), as finanças do usuário: saldo do período, saldo previsto, totais de receitas e despesas, contas pendentes, gastos por categoria e lançamentos

	recentes.
Ator Primário	Usuário
Atores Secundários	—
Pré-condição	O usuário está autenticado no sistema.
Cenário Principal	1. O use case inicia quando o usuário acessa o sistema ou seleciona a opção "Visão geral" no menu lateral. 2. O sistema identifica o período selecionado (por padrão, todos os períodos). 3. O sistema calcula o total de receitas e o total de despesas do período (RN002). 4. O sistema calcula o saldo do período (receitas menos despesas pagas – RN001) e o saldo previsto (receitas menos todas as despesas, inclusive as ainda a pagar). 5. O sistema calcula a quantidade e o valor total das contas pendentes. 6. O sistema agrupa as despesas por categoria e gera o gráfico de gastos por categoria (RN004). 7. O sistema exibe os indicadores, o gráfico de evolução do saldo e os lançamentos recentes. 8. O usuário pode alterar o período (mês/ano); o sistema retorna ao passo 3. 9. O caso de uso termina quando o usuário navega para outra tela.
Pós-condição	O balanço financeiro do período foi calculado e apresentado ao usuário.
Cenário Alternativo	3a – Não há lançamentos no período 3a.1 O sistema exibe os indicadores zerados e a mensagem "Nenhuma despesa no período selecionado" no lugar do gráfico. 7a – Usuário aciona "Novo lançamento" 7a.1 O sistema executa o caso de uso Cadastrar Despesa (UC002) ou Cadastrar Receita (UC003), conforme o tipo escolhido, e retorna ao passo 3.

Fonte: elaborado pelos autores (2026).

Quadro 9 – Use Case Manter Categoria

Caso de Uso – Manter Categoria	
ID	UC006
Descrição	Este caso de uso tem por objetivo manter (incluir, consultar, alterar e excluir) as categorias de despesas e receitas do usuário, com nome e cor personalizados. Pode ser acionado diretamente pelo menu ou, por meio do relacionamento «extend», a partir de Manter Despesa (UC002) e de Manter Receita (UC003).
Ator Primário	Usuário
Atores Secundários	—
Pré-condição	O usuário está autenticado no sistema.
Cenário Principal	1. O use case inicia quando o usuário seleciona a opção "Categorias" no

	menu lateral ou quando é acionado, por extensão («extend»), pela opção "Nova categoria" de Manter Despesa (UC002) ou de Manter Receita (UC003). 2. O sistema apresenta a lista de categorias cadastradas, agrupadas por tipo, com a quantidade e o total de lançamentos de cada uma. 3. O usuário aciona a opção "Nova categoria". 4. O sistema carrega o formulário de categoria. 5. O usuário informa o tipo (despesa ou receita), o nome e a cor da categoria. 6. O usuário confirma o cadastro. 7. O sistema valida os dados (nome obrigatório e não repetido para o mesmo tipo). 8. O sistema grava a categoria, exibe mensagem de sucesso e retorna à lista. 9. Se o caso de uso foi acionado por extensão a partir de um lançamento, o sistema retorna ao caso de uso de origem (UC002 ou UC003); caso contrário, o caso de uso termina.
Pós-condição	Uma categoria foi incluída, consultada, alterada ou excluída.
Cenário Alternativo	2a – Consulta de categorias 2a.1 O usuário filtra as categorias por tipo (despesa ou receita). 2a.2 O sistema exibe as categorias correspondentes, com a quantidade e o total de lançamentos de cada uma, e o caso de uso termina. 3a – Alteração 3a.1 O usuário seleciona uma categoria e aciona "Editar". 3a.2 O sistema carrega o formulário preenchido; o usuário altera nome e/ou cor e confirma; o caso de uso continua no passo 7. 3b – Exclusão 3b.1 O usuário seleciona uma categoria e aciona "Excluir". 3b.2 O sistema verifica se há lançamentos vinculados (RN008). Se não houver, solicita confirmação e exclui; caso contrário, informa que a categoria possui lançamentos e não realiza a exclusão. 3c – Favoritar ou reordenar 3c.1 O usuário marca a categoria como favorita ou arrasta-a para outra posição; o sistema salva a preferência e retorna ao passo 2. 6a – Usuário cancela 6a.1 O sistema descarta os dados e retorna ao passo 2. 7a – Nome em branco ou já existente 7a.1 O sistema informa o erro e retorna ao passo 5.

Fonte: elaborado pelos autores (2026).

Quadro 10 – Use Case Alterar Status da Despesa

Caso de Uso – Alterar Status da Despesa	
ID	UC007
Descrição	Este caso de uso estende Listar Despesas (UC004), por meio do relacionamento «extend», e permite ao usuário marcar uma despesa como paga ou voltá-la para a situação "A pagar" com um único clique.

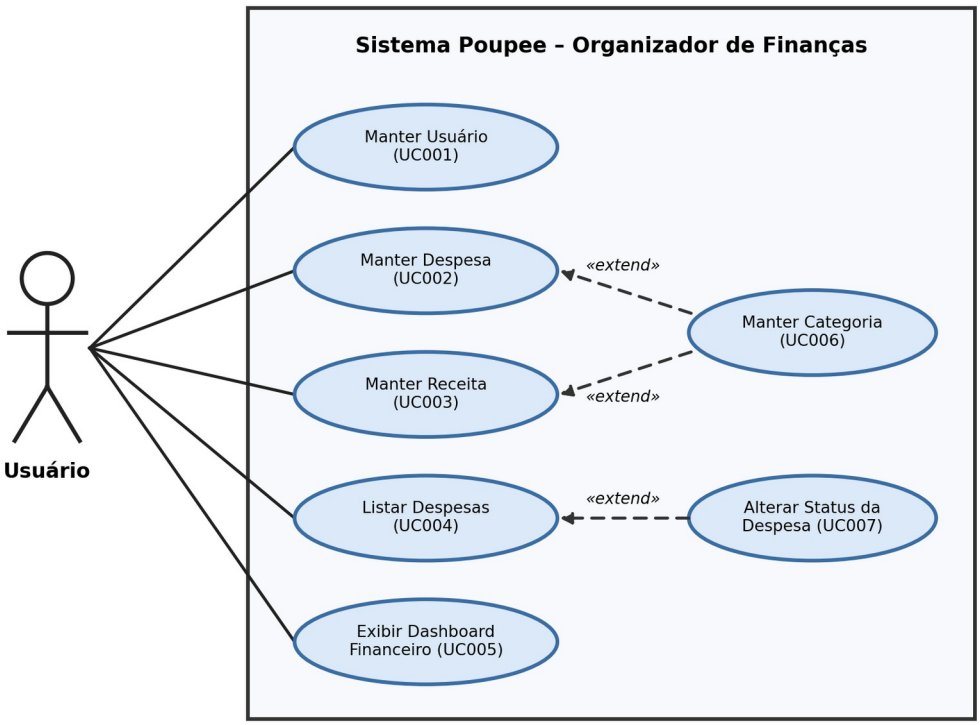
Ator Primário	Usuário
Atores Secundários	—
Pré-condição	O usuário está autenticado e visualizando a lista de despesas (UC004).
Cenário Principal	1. O use case inicia, a partir do ponto de extensão do passo 6 de Listar Despesas (UC004), quando o usuário aciona o controle de status (interruptor) de uma despesa na lista. 2. O sistema inverte o status da despesa: de "A pagar" para "Paga" ou de "Paga" para "A pagar" (RN006). 3. O sistema grava a alteração. 4. O sistema recalcula o total de contas pendentes e o balanço do período (RN001). 5. O sistema atualiza a lista de acordo com o filtro ativo e retorna ao caso de uso Listar Despesas.
Pós-condição	O status da despesa foi alterado e os indicadores foram atualizados.
Cenário Alternativo	3a – Falha ao gravar a alteração 3a.1 O sistema informa o erro, mantém o status anterior e retorna ao caso de uso Listar Despesas.

Fonte: elaborado pelos autores (2026).

2.7 Diagrama de Caso de Uso

A Figura 2 apresenta o diagrama de casos de uso do sistema, com o ator identificado e os relacionamentos de extensão («extend») entre os casos de uso: Manter Categoria estende Manter Despesa e Manter Receita, e Alterar Status da Despesa estende Listar Despesas. Em cada caso de uso estendido, o ponto de extensão está indicado no cenário principal e o fluxo da extensão é descrito no cenário alternativo correspondente.

Figura 2 – Diagrama de Casos de Uso do sistema

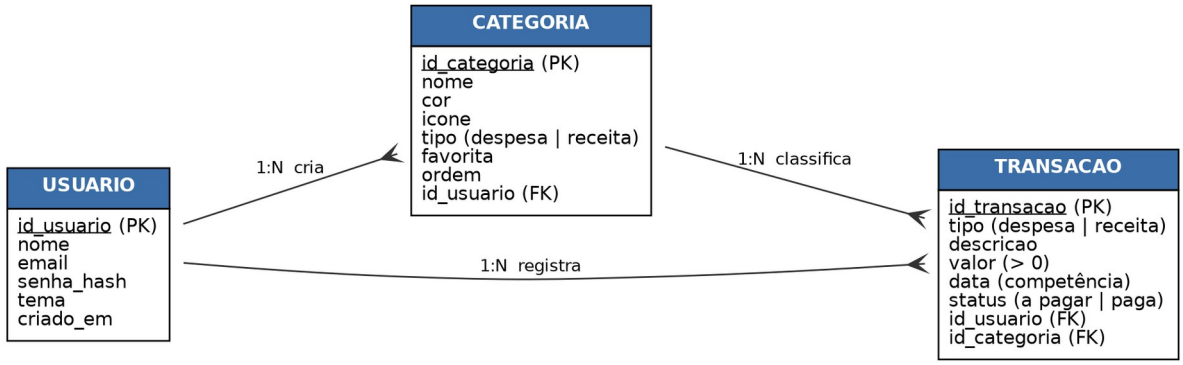


Fonte: elaborado pelos autores (2026).

3.0 – Diagrama Entidade Relacionamento

A Figura 3 apresenta o modelo entidade-relacionamento proposto. Um usuário possui várias categorias e vários lançamentos (transações); cada categoria pertence a um único usuário e é do tipo despesa ou receita; cada transação pertence a um usuário e é classificada por exatamente uma categoria, que deve ser do mesmo tipo da transação (RN005). O campo "data" representa a data de competência (RN002), o campo "valor" deve ser sempre positivo (RN003) e o campo "status" só se aplica às despesas (RN006). O balanço mensal (RN001) é obtido por uma consulta que agrupa as transações por usuário, ano e mês. A implementação física desse modelo no PostgreSQL é descrita na seção 5.3.

Figura 3 – Diagrama Entidade-Relacionamento



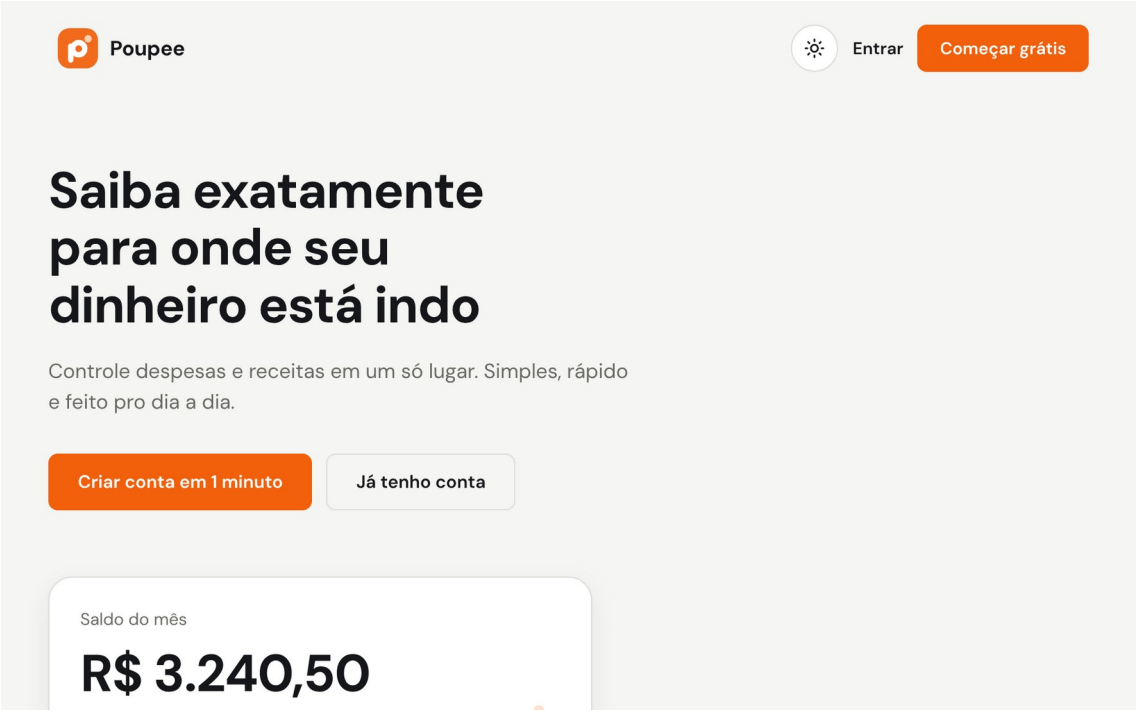
Fonte: elaborado pelos autores (2026).

4.0 – Telas da aplicação

As telas foram desenvolvidas para ambiente web em HTML, CSS e JavaScript, a partir do protótipo navegável, e hoje estão ligadas à API do sistema: tudo o que o usuário cadastra fica gravado no banco de dados. A aplicação contempla a página inicial de apresentação (landing page), as telas de login e criação de conta, a Visão Geral com o balanço financeiro, as listas de despesas e receitas, o cadastro de categorias e o formulário de novo lançamento, além da alternância entre os temas claro e escuro. O sistema está publicado e pode ser acessado pelo link a seguir (login de exemplo: thiago@exemplo.com, senha 123456):

Acesse o Poupee: https://poupee-pi.vercel.app

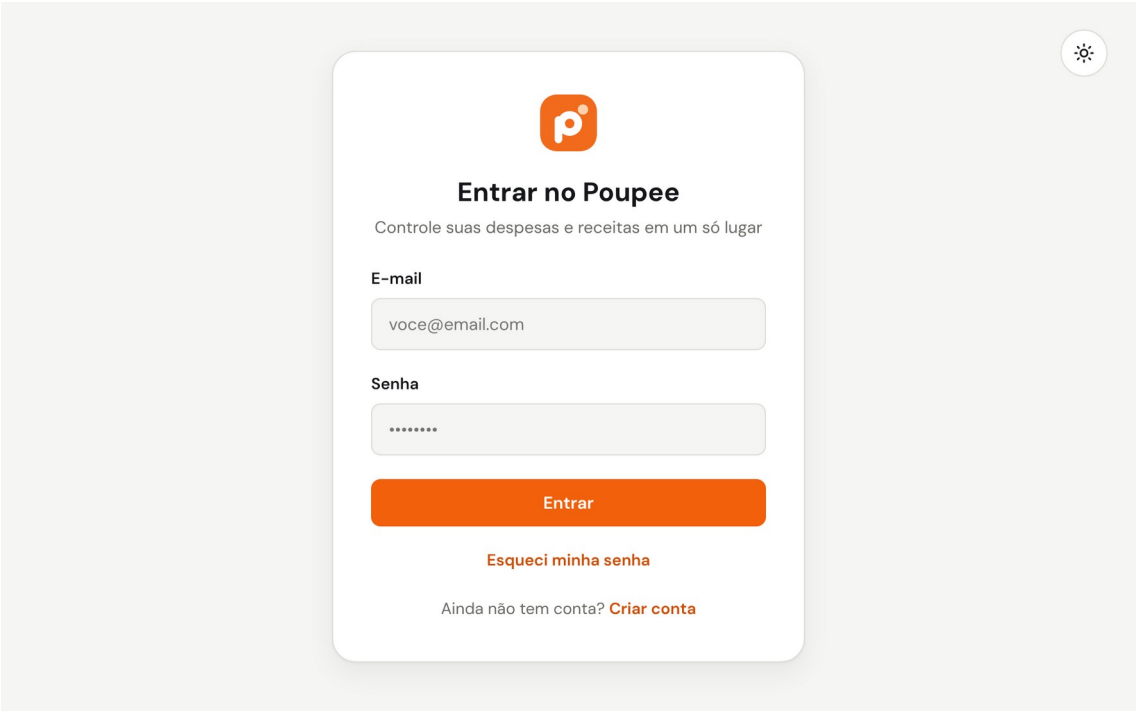
Figura 4 – Página inicial (Landing Page)



Apresentação do Poupee, com o problema que ele resolve, os benefícios e os atalhos "Criar conta" e "Entrar".

Fonte: elaborado pelos autores (2026).

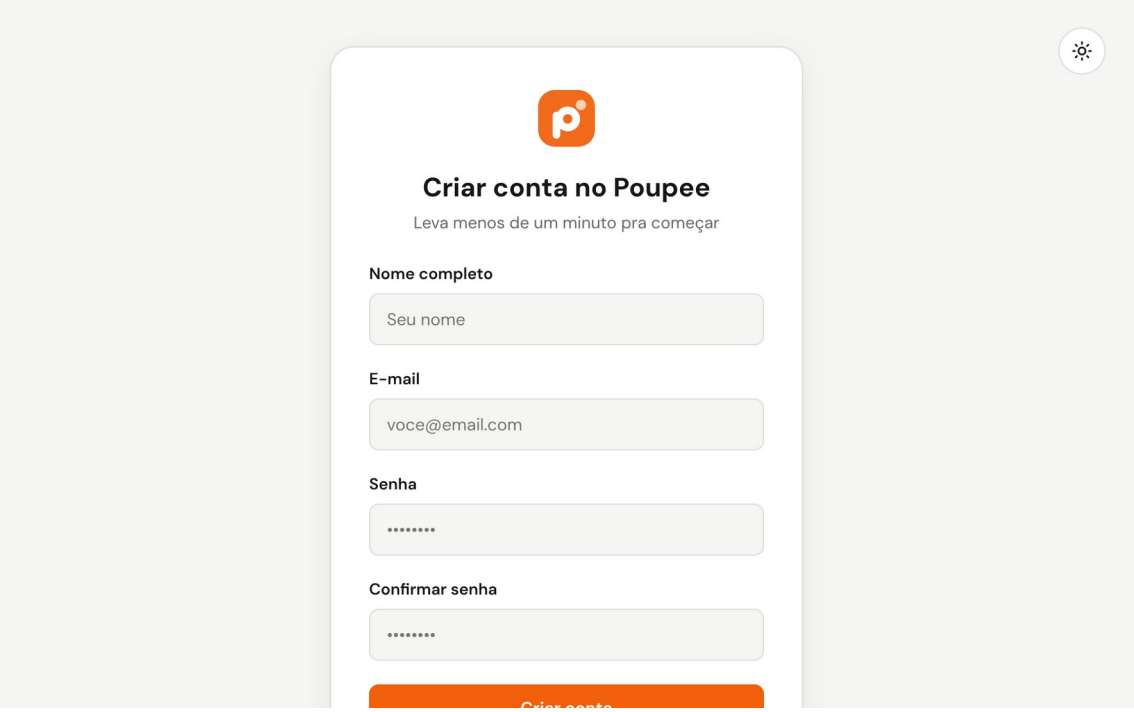
Figura 5 – Tela de Login



Acesso ao sistema com e-mail e senha, link para recuperação de senha e atalho para criação de conta.

Fonte: elaborado pelos autores (2026).

Figura 6 – Tela de Criar Conta



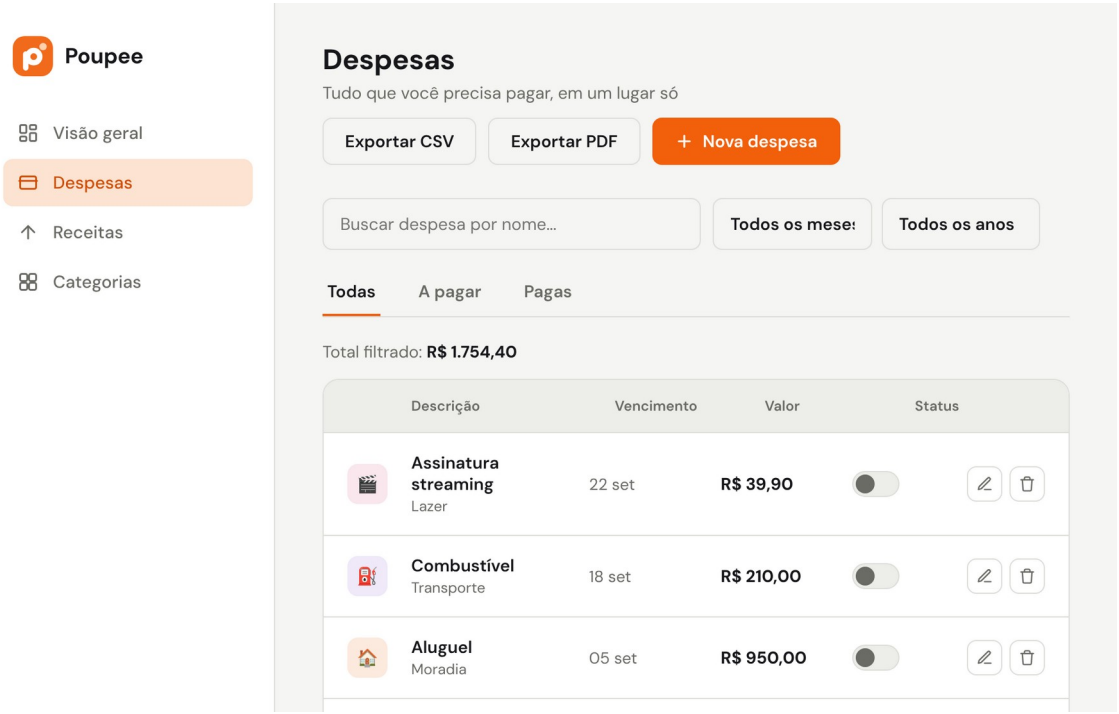
Cadastro do usuário com nome completo, e-mail, senha e confirmação de senha.
Fonte: elaborado pelos autores (2026).

Figura 7 – Tela de Visão Geral (Balanço Financeiro)



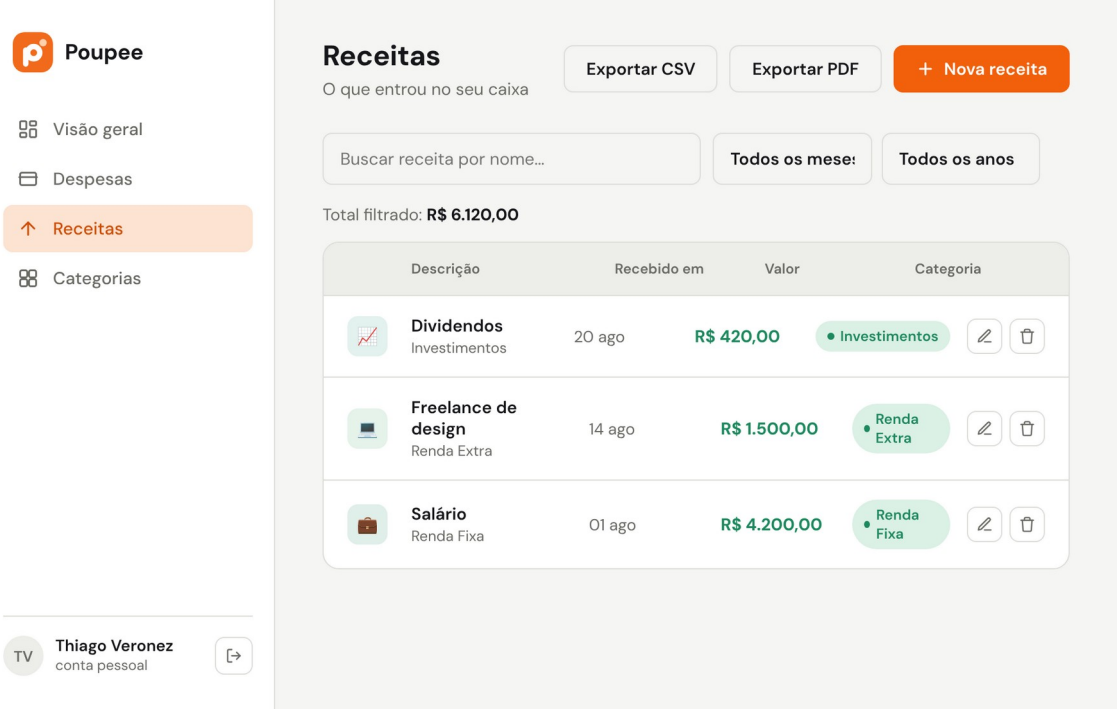
Saldo do período, saldo previsto, totais de receitas e despesas, contas pendentes, gráfico de gastos por categoria e lançamentos recentes, com filtro de mês/ano.
Fonte: elaborado pelos autores (2026).

Figura 8 – Tela de Despesas



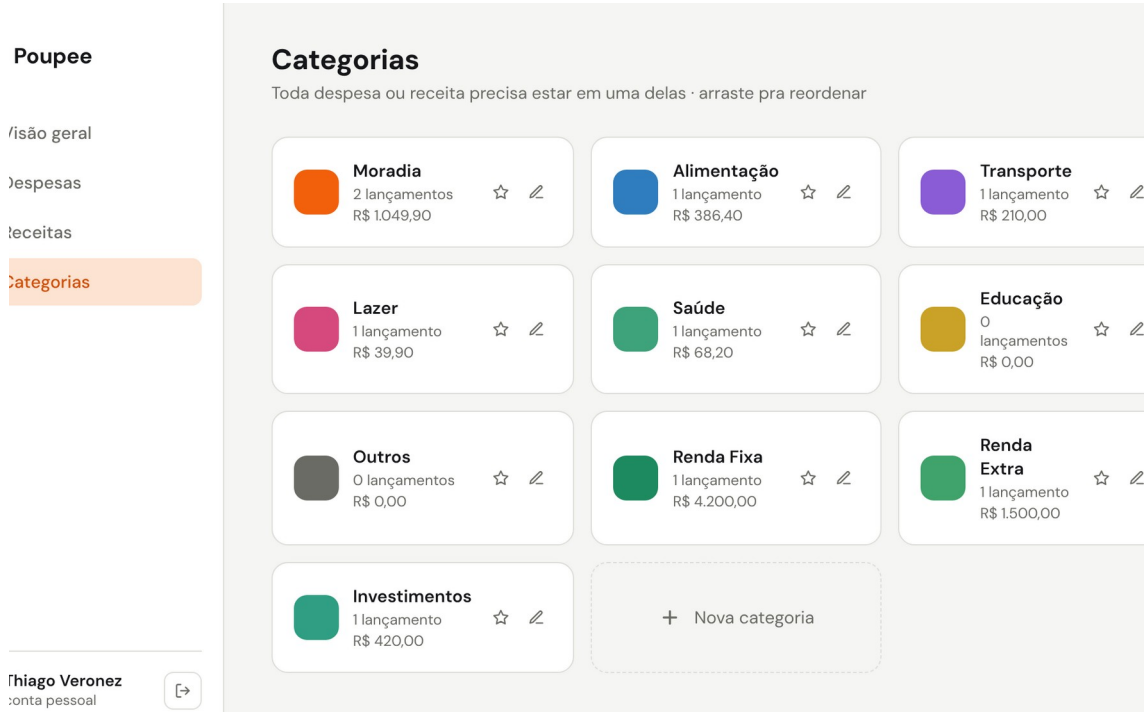
Lista de despesas com filtros "Todas", "A pagar" e "Pagas", busca por descrição, total filtrado, interruptor de status e ações de editar/excluir, além da exportação em CSV e PDF.
Fonte: elaborado pelos autores (2026).

Figura 9 – Tela de Receitas



Lista de receitas com data de recebimento, valor e categoria, filtro por período e exportação.
Fonte: elaborado pelos autores (2026).

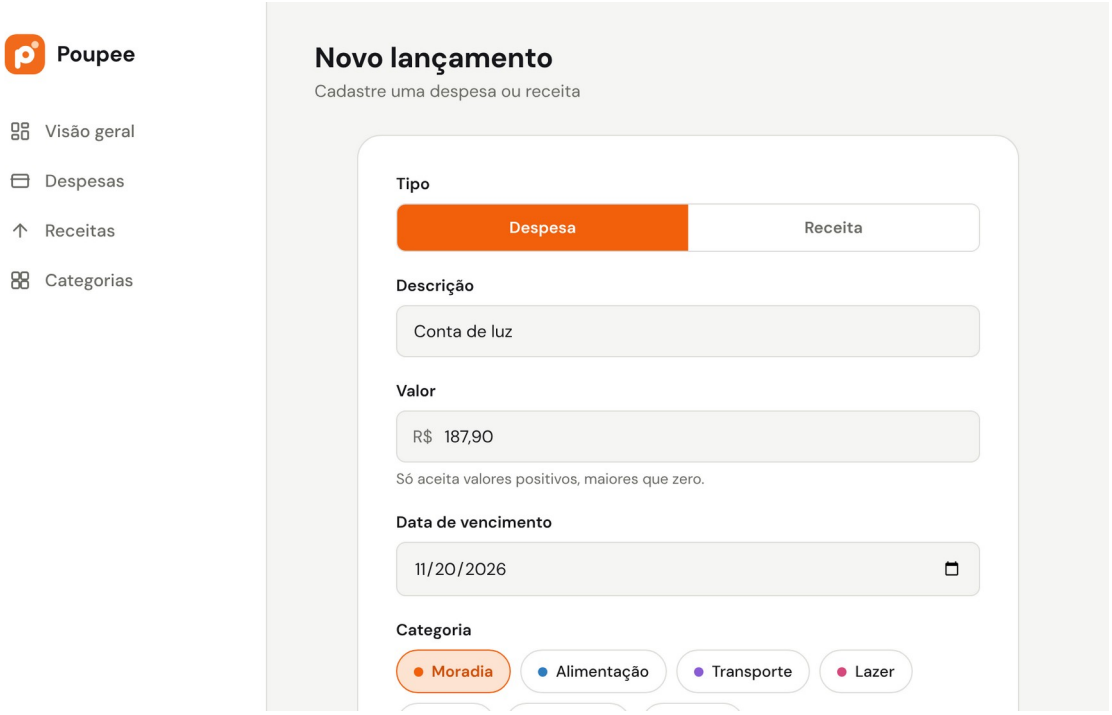
Figura 10 – Tela de Categorias



Cadastro de categorias de despesas e receitas com nome e cor, favoritos e reordenação por arrastar e soltar.

Fonte: elaborado pelos autores (2026).

Figura 11 – Tela de Novo Lançamento



Formulário único para despesas e receitas: tipo, descrição, valor (somente positivo), data, categoria e status.

Fonte: elaborado pelos autores (2026).

5.0 – Arquitetura e Implementação

Concluída a modelagem, o Poupee foi implementado como uma aplicação web completa, composta por front-end, API REST e banco de dados relacional, e publicado na internet. Esta seção descreve a arquitetura adotada, as tecnologias utilizadas, a implementação do banco de dados, a API e as medidas de segurança.

5.1 Arquitetura do sistema

O sistema segue a arquitetura cliente-servidor, organizada em camadas (Figura 12). O front-end é executado no navegador do usuário e se comunica com a API por requisições HTTPS que trocam dados no formato JSON. A API, escrita em Node.js com o framework Express, é dividida em quatro camadas com responsabilidades separadas: as rotas recebem as requisições, conferem o token de acesso e devolvem as respostas; os serviços aplicam as regras que dependem de dados já gravados, como verificar se a categoria pertence ao usuário e é do mesmo tipo do lançamento; as regras reúnem funções puras de validação, filtros por período e cálculo do balanço, que não dependem do banco e por isso são fáceis de testar; e os repositórios concentram todo o acesso ao banco de dados, sendo o único ponto do código que contém comandos SQL.

Essa separação aplica o padrão de projeto Repository: os serviços conversam com uma interface de repositório e não sabem qual banco está por trás dela. Há duas implementações dessa interface, uma para o PostgreSQL e outra em memória, e a escolha é feita na montagem da aplicação (injeção de dependência), conforme as variáveis de ambiente. A versão em memória é usada nos testes automatizados e no modo de demonstração, em que o sistema funciona sem banco configurado.

Figura 12 – Arquitetura do sistema



Fonte: elaborado pelos autores (2026).

5.2 Tecnologias utilizadas

O Quadro 11 relaciona as tecnologias empregadas em cada camada do sistema e a finalidade de cada uma.

Quadro 11 – Tecnologias utilizadas

Camada	Tecnologia	Uso no projeto
Front-end	HTML5, CSS3 e JavaScript	Estrutura, aparência e comportamento das telas; máscara de valores; chamadas à API com fetch
Front-end	Chart.js 4	Gráfico de gastos por categoria e evolução do saldo na Visão Geral
Front-end	jsPDF	Exportação das listas de despesas e receitas em PDF
Front-end	Driver.js	Tour de boas-vindas no primeiro acesso
Back-end	Node.js 18+ e Express 4	API REST com as rotas do sistema
Back-end	bcryptjs	Criptografia (hash) das senhas
Back-end	jsonwebtoken (JWT)	Autenticação: emissão e conferência do token de acesso
Back-end	pg	Driver de conexão com o PostgreSQL
Banco de dados	PostgreSQL 16 (Neon)	Armazenamento de usuários, categorias e lançamentos
Hospedagem	Vercel	Publicação do front-end e execução da API como função serverless
Versionamento	Git e GitHub	Controle de versão, Pull Requests e revisão de código
Qualidade	GitHub Actions e node:test	Execução automática dos testes a cada alteração

Fonte: elaborado pelos autores (2026).

5.3 Banco de dados relacional

O SGBD adotado é o PostgreSQL, hospedado no serviço Neon, na região de São Paulo. A especificação inicial previa o MySQL; a troca foi decidida pela equipe

durante a implementação porque o Neon oferece um plano gratuito integrado à Vercel, o que simplificou a publicação, e porque o modelo continua sendo relacional, com as mesmas tabelas, chaves e restrições. O requisito RNF003 foi atualizado para refletir essa decisão.

O modelo lógico, derivado do diagrama entidade-relacionamento da seção 3, é composto por três tabelas, apresentadas no Quadro 12. As despesas e as receitas ficam na mesma tabela, transacoes, diferenciadas pela coluna tipo, o que simplifica o cálculo do balanço e as consultas por período.

Quadro 12 – Modelo lógico do banco de dados

Tabela	Colunas	Chaves e restrições
usuarios	id, nome, email, senha_hash, tema, criado_em, atualizado_em	PK id; e-mail único (RN007); tema "light" ou "dark"
categorias	id, usuario_id, nome, cor, icone, tipo, favorita, ordem, criado_em	PK id; FK usuario_id → usuarios; tipo "despesa" ou "receita"; cor em hexadecimal; nome único por usuário e tipo
transacoes	id, usuario_id, categoria_id, tipo, descricao, valor NUMERIC(10,2), data, status, criado_em, atualizado_em	PK id; FK usuario_id → usuarios; FK categoria_id → categorias (RESTRICT, RN008); valor > 0 (RN003); status "pagar" ou "paga"

Fonte: elaborado pelos autores (2026).

Além das tabelas, o banco possui objetos programados que garantem as regras de negócio diretamente no SGBD: o gatilho trg_transacoes_categoria_tipo impede gravar um lançamento em uma categoria de outro tipo (RN005); os gatilhos de atualização preenchem a coluna atualizado_em a cada alteração; a visão vw_balanco_mensal calcula, por usuário, ano e mês, os totais de receitas e despesas, o valor pendente, o saldo realizado e o saldo previsto (RN001); e índices sobre usuário e data aceleram as consultas filtradas por período. Os scripts de criação estão em db/schema.sql e os dados de exemplo em db/seed.sql. Quando a API encontra o banco vazio, ela executa esses scripts automaticamente.

As regras de negócio são verificadas em duas camadas, na API e no banco, de modo que um erro em uma delas não comprometa os dados. O Quadro 13 mostra onde cada regra é aplicada.

Quadro 13 – Onde cada regra de negócio é aplicada

Regra	Na API	No banco de dados
RN001 – Balanço do período	Cálculo em centavos em regras/balanco.js (rota /api/balanco)	Visão vw_balanco_mensal
RN002 – Data de	Filtros por mês e ano sobre a data	Coluna data obrigatória e

Regra	Na API	No banco de dados
competência	do lançamento	índice por usuário e data
RN003 – Valores positivos	Validação: maior que zero e até duas casas decimais	CHECK (valor > 0)
RN004 – Categoria obrigatória	Validação do campo categoriaId	categoria_id NOT NULL com chave estrangeira
RN005 – Mesmo tipo	Serviço confere o tipo da categoria	Gatilho trg_transacoes_categoria_tipo
RN006 – Status só em despesas	Receita é sempre gravada como recebida	CHECK nos valores de status
RN007 – E-mail único	Resposta 409 com mensagem ao usuário	Restrição UNIQUE em email
RN008 – Exclusão de categoria	Resposta 409 se houver lançamentos	Chave estrangeira com ON DELETE RESTRICT

Fonte: elaborado pelos autores (2026).

5.4 API REST

A API segue o estilo REST: cada recurso do sistema (usuário, categorias, transações e balanço) possui um endereço próprio, e a operação é indicada pelo método HTTP. As respostas são sempre em JSON e usam os códigos de status do protocolo: 200 e 201 para sucesso, 400 para dados inválidos (com a mensagem de cada campo), 401 para acesso sem login, 404 para registro não encontrado e 409 para conflitos, como e-mail já cadastrado ou categoria em uso. O Quadro 14 lista os endpoints; a documentação completa, com exemplos de requisição e resposta, está no arquivo backend/README.md do repositório.

Quadro 14 – Endpoints da API

Método	Rota	Descrição
POST	/api/usuarios	Cria a conta (com as categorias padrão)
POST	/api/login	Autentica e devolve o token de acesso
GET / PUT	/api/eu	Consulta e altera nome e tema do usuário logado
GET / POST	/api/categorias	Lista (com filtro por tipo) e cria categorias
GET / PUT / DELETE	/api/categorias/:id	Consulta, altera e exclui uma categoria
GET / POST	/api/transacoes	Lista com filtros (tipo, mês, ano, status, busca) e cria lançamentos
GET / PUT / DELETE	/api/transacoes/:id	Consulta, altera e exclui um lançamento
PATCH	/api/transacoes/:id/status	Marca a despesa como paga ou a pagar
GET	/api/balanco	Balanço do período: totais, saldos, pendências e gastos por categoria
GET	/api/saude	Verifica se a API está no ar e se está usando o banco

Fonte: elaborado pelos autores (2026).

5.5 Segurança

As medidas de segurança adotadas atendem ao requisito RNF005:

- **Senhas protegidas –** as senhas são gravadas apenas como hash bcrypt; nem a equipe nem o banco conhecem a senha original.
- **Autenticação por token –** após o login, a API emite um token JWT válido por sete dias, enviado em todas as requisições; sem ele, a API responde 401 e o front volta para a tela de login.
- **Isolamento dos dados –** todas as consultas são filtradas pelo identificador do usuário do token, de modo que um usuário nunca acessa dados de outro.
- **Consultas parametrizadas –** os valores enviados pelo usuário nunca são concatenados ao SQL, o que impede ataques de injeção de SQL.
- **Segredos fora do código –** o endereço do banco (DATABASE_URL) e a chave do token (JWT_SEGREDO) ficam em variáveis de ambiente da Vercel e não são publicados no repositório.
- **Conexões criptografadas –** o acesso ao site usa HTTPS e a conexão da API com o banco no Neon usa SSL.

6.0 – Implantação, Versionamento e Testes

6.1 Hospedagem

O sistema está publicado na Vercel, no endereço https://poupee-pi.vercel.app. O mesmo projeto da Vercel serve os arquivos do front-end e executa a API como função serverless (arquivo api/index.js), de modo que front e API compartilham o mesmo endereço e não há configuração extra de domínio. O banco de dados poupee-db, no Neon, foi conectado ao projeto pela integração de Storage da Vercel, que cria automaticamente a variável DATABASE_URL.

A publicação é contínua: cada alteração aprovada e mesclada na branch main gera uma nova versão do site em poucos minutos, e cada Pull Request recebe um endereço de pré-visualização para conferência antes do merge. Para verificar se o sistema está no ar e conectado ao banco, basta acessar https://poupee-pi.vercel.app/api/saude, que responde "modo": "postgres".

6.2 Controle de versão

O código está versionado no GitHub, no repositório PouPeePI/Projeto-financas. A equipe segue um fluxo baseado em Pull Requests: cada mudança é feita em uma branch própria, enviada como Pull Request, revisada e mesclada na main somente depois que os testes automáticos passam. As convenções do time (nomes de branch, mensagens de commit e passo a passo do Git) estão no arquivo CONTRIBUTING.md,

e o README.md apresenta o projeto, as funcionalidades, as telas e as instruções de execução.

6.3 Testes automatizados

O back-end possui 27 testes automatizados, escritos com o executor de testes nativo do Node.js (node:test). Eles cobrem as regras de negócio, como validação de valores e datas, filtros por período e cálculo do balanço, e o funcionamento da API de ponta a ponta: criação de conta, login, CRUD de categorias e lançamentos, alteração de status, isolamento entre usuários e respostas de erro. A cada Pull Request, o GitHub Actions executa os testes duas vezes, uma com o repositório em memória e outra contra um banco PostgreSQL 16 real, garantindo que o SQL também funciona.

6.4 Como executar o projeto

Para executar o sistema em um computador, é necessário ter o Node.js 18 ou mais recente instalado. Os passos são:

1. clonar o repositório: git clone https://github.com/PouPeePI/Projeto-financas.git;

2. entrar na pasta backend e instalar as dependências com npm install;

3. executar npm run dev:memoria para usar dados de exemplo, ou copiar o arquivo .env.example para .env, informar o endereço de um banco PostgreSQL e executar npm start;

4. abrir http://localhost:3000 no navegador e entrar com thiago@exemplo.com e a senha 123456.

7.0 – Links do Projeto

Quadro 15 – Links do projeto

Item	Endereço
Sistema publicado	https://poupee-pi.vercel.app
Repositório no GitHub	https://github.com/PouPeePI/Projeto-financas
Documentação da API	https://github.com/PouPeePI/Projeto-financas/blob/main/backend/README.md
Vídeo de demonstração (YouTube)	https://youtu.be/EKjECa7p4eM

Fonte: elaborado pelos autores (2026).

Conclusão

Este trabalho apresentou a análise, a modelagem, a implementação e a implantação do Poupee, um organizador de finanças pessoais para ambiente web. A

partir do problema identificado — a dificuldade de saber para onde vai o dinheiro ao longo do mês — foram definidos um ator, sete requisitos funcionais, sete requisitos não funcionais e oito regras de negócio, que deram origem aos casos de uso, ao processo de negócio em BPMN, ao modelo entidade-relacionamento e ao protótipo das telas. Em seguida, o sistema foi implementado com front-end em HTML, CSS e JavaScript, API REST em Node.js e banco de dados relacional PostgreSQL, e publicado na Vercel, onde pode ser usado por qualquer pessoa.

A descrição dos casos de uso com cenários alternativos permitiu antecipar situações reais de uso — valor inválido, categoria inexistente, período sem lançamentos, exclusão de categoria com vínculos — que foram tratadas tanto na API quanto no banco de dados. A organização do back-end em camadas e os testes automatizados executados a cada alteração deram segurança para evoluir o sistema, inclusive na troca do SGBD durante o projeto. Como trabalhos futuros, sugere-se a recuperação de senha por e-mail, lembretes automáticos de contas a vencer, lançamentos recorrentes, metas de gastos por categoria e o controle de contas bancárias.